Lambda Functions.

What Lambda actually is

It's a **serverless compute service** where you:

- Upload code (Python, Node.js, etc.)
- AWS runs it **only when triggered**
- You don't manage EC2, OS, scaling, nothing

Simple analogy (no fluff)

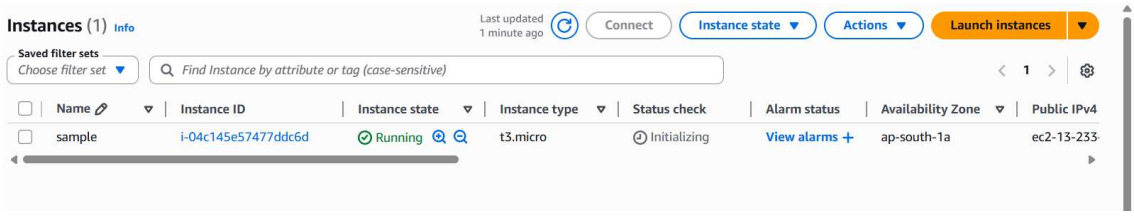
- EC2 → You rent a machine and manage everything
- Lambda → You just give a task, AWS runs it and disappears

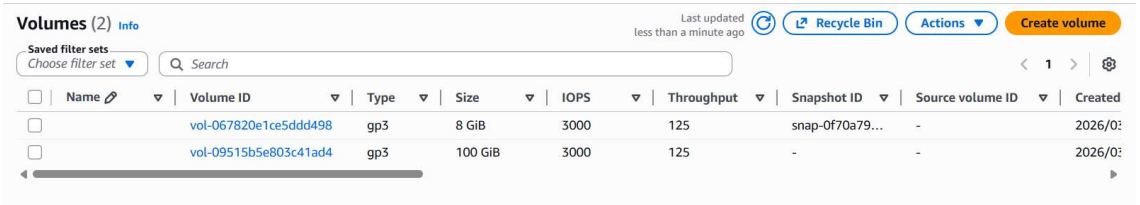
How it works (real flow)

1. You write a function
2. You upload it to Lambda
3. Something triggers it:
    - API call
    - File upload to S3
    - Timer (cron job)
4. Lambda runs → finishes → stops

I have created an Lambda Function To delete the EC2 and EBS,Elastic IP.

I have created an EC2 and EBS Volume And Attached to that instance.

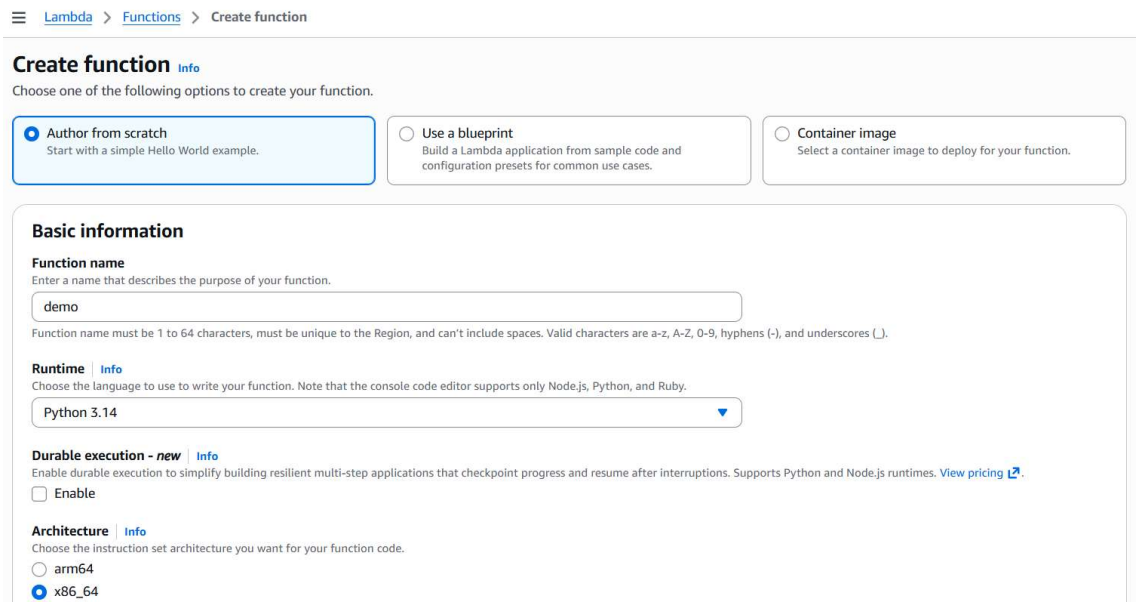




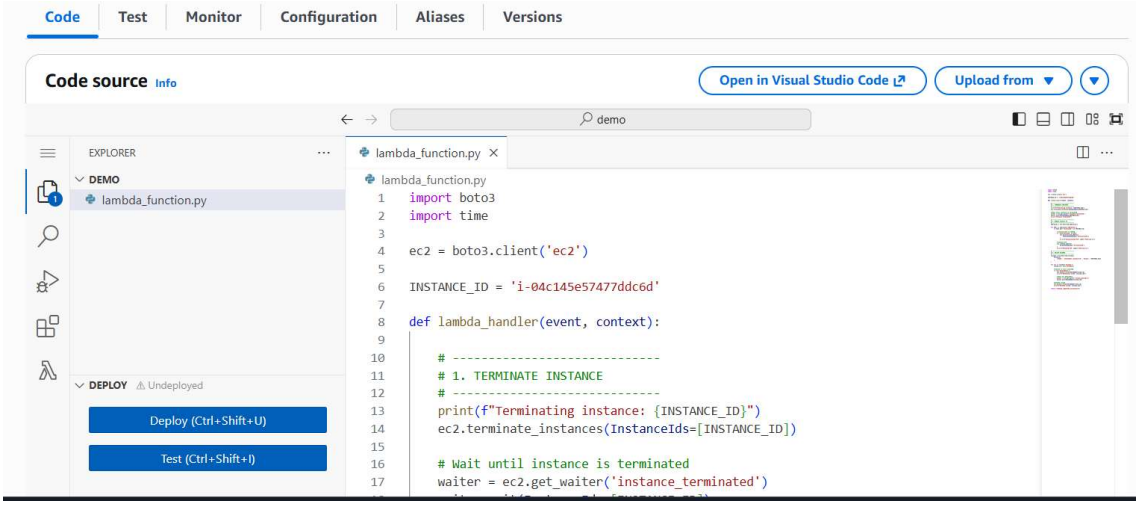
I have created a Elastic IP and associste with our instance

Then I have Created An Lambda Functions.

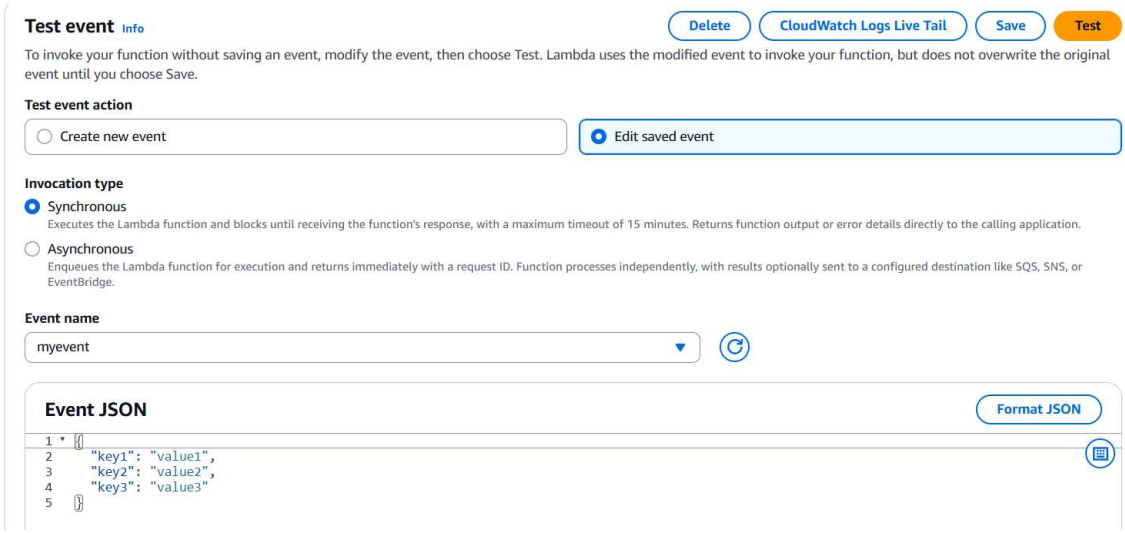
Then I have Created an function With the Python Runtime Environment.



Then I have deployed the code.



The Testing Phase.



Command to Delete and Instance And Detach From the Instance From the EBS Volume, Deassociate the Elastic IP

import boto3

import time

ec2 = boto3.client('ec2')

INSTANCE_ID = 'i-04c145e57477ddc6d'

```
def lambda_handler(event, context):

    # -----------------------------
    # 1. TERMINATE INSTANCE
    # -----------------------------
    print(f"Terminating instance: {INSTANCE_ID}")
    ec2.terminate_instances(InstanceIds=[INSTANCE_ID])

    # Wait until instance is terminated
    waiter = ec2.get_waiter('instance_terminated')
    waiter.wait(InstanceIds=[INSTANCE_ID])
    print("Instance terminated")

    # -----------------------------
    # 2. HANDLE ELASTIC IP
    # -----------------------------
    addresses = ec2.describe_addresses()

    for addr in addresses['Addresses']:
        if addr.get('InstanceId') == INSTANCE_ID:

            # Disassociate if needed
            if 'AssociationId' in addr:
                ec2.disassociate_address(
                    AssociationId=addr['AssociationId']
                )
                print(f"Disassociated EIP: {addr['PublicIp']}")

            # Release EIP
```

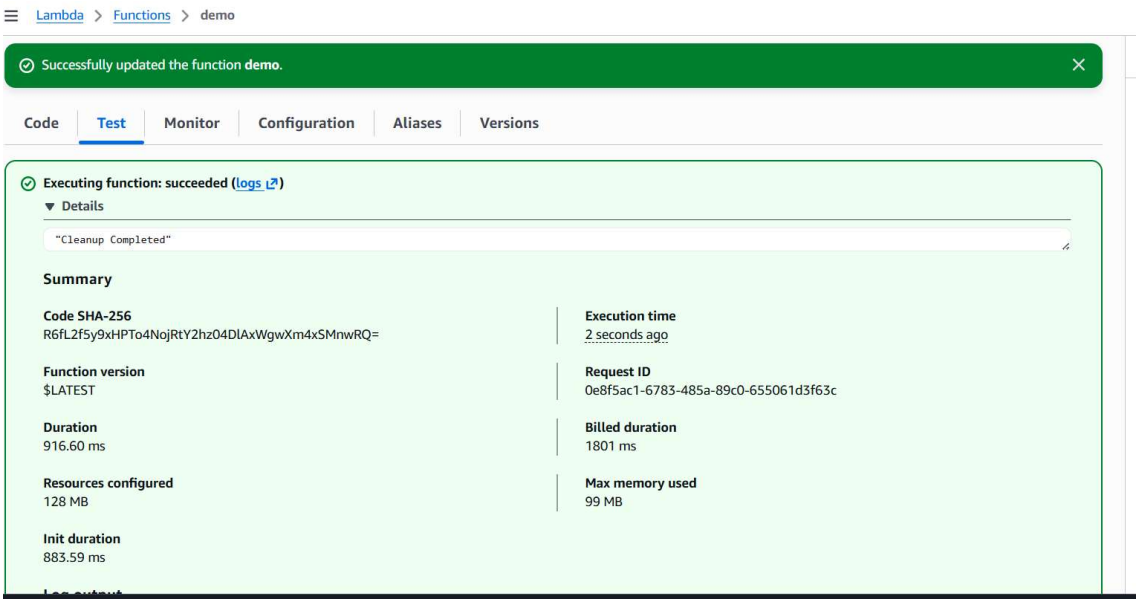
```
        ec2.release_address(
            AllocationId=addr['AllocationId']
        )
        print(f"Released EIP: {addr['PublicIp']}")

# -----------------------------
# 3. DELETE VOLUMES
# -----------------------------
volumes = ec2.describe_volumes(
    Filters=[
        {'Name': 'attachment.instance-id', 'Values': [INSTANCE_ID]}
    ]
)

for vol in volumes['Volumes']:
    volume_id = vol['VolumeId']

    # Detach if still attached
    if vol['Attachments']:
        ec2.detach_volume(VolumeId=volume_id)
        print(f"Detaching volume: {volume_id}")

        # Wait for detachment
        waiter = ec2.get_waiter('volume_available')
        waiter.wait(VolumeIds=[volume_id])

    # Delete volume
    ec2.delete_volume(VolumeId=volume_id)
    print(f"Deleted volume: {volume_id}")
```

```
    return "Cleanup completed successfully"
```

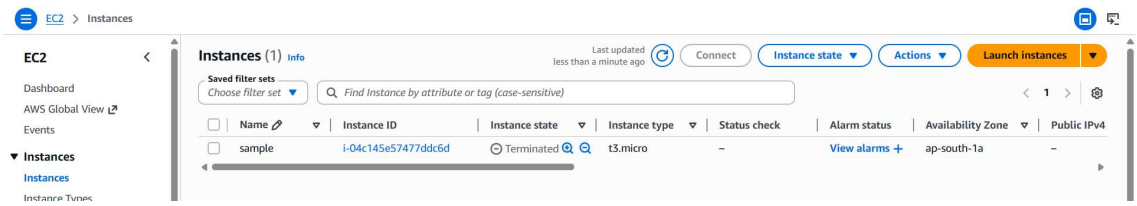
And then deleting the Detached Volume and release the Elastic IP

```
import boto3

ec2 = boto3.client('ec2')

def lambda_handler(event, context):

    # ------------------------------
    # 1. RELEASE ELASTIC IP
    # ------------------------------
    addresses = ec2.describe_addresses()

    for addr in addresses['Addresses']:
        try:
            # Disassociate if still associated
            if 'AssociationId' in addr:
                ec2.disassociate_address(
                    AssociationId=addr['AssociationId']
                )
                print(f"Disassociated: {addr['PublicIp']}")

            # Release EIP
            ec2.release_address(
                AllocationId=addr['AllocationId']
            )
            print(f"Released EIP: {addr['PublicIp']}")
```

```
    except Exception as e:
        print(f"EIP Error: {str(e)}")

# ----------------------------
# 2. DELETE AVAILABLE VOLUMES
# ----------------------------
volumes = ec2.describe_volumes(
    Filters=[
        {'Name': 'status', 'Values': ['available']}
    ]
)

for vol in volumes['Volumes']:
    volume_id = vol['VolumeId']

    try:
        ec2.delete_volume(VolumeId=volume_id)
        print(f"Deleted Volume: {volume_id}")
    except Exception as e:
        print(f"Volume Error: {str(e)}")

return "Cleanup Completed"
```

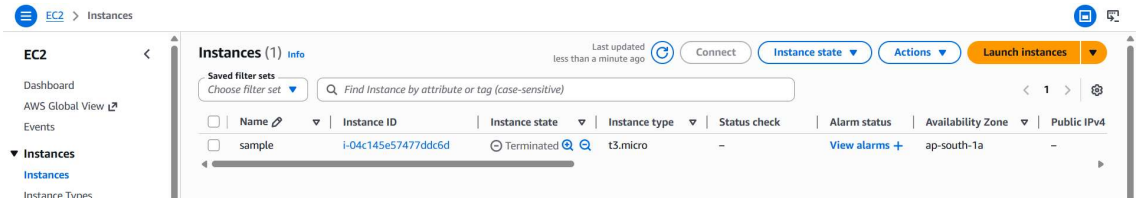


Terminated Instance



Detached Volume

Then deleted the volume.



The deassociated ElasticIP